



Report

Kriti '26

Bot Design Challenge

Hostel ID:

2601

keus[®]

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Objectives	3
2	System Overview	4
2.1	Overall System Description	4
2.2	Working Principle of the Rover	4
2.3	System Architecture	4
2.4	Hardware–Software Interaction	5
3	Hardware Design	5
3.1	Motor and Drive System	5
3.2	Battery and Power System	5
3.3	Sensors and Perception Hardware	6
3.4	Camera and Image Processing System	6
3.5	Communication Modules	7
3.6	Hardware Integration	7
4	Circuit Design	7
4.1	Circuit Architecture	7
4.2	Microcontroller / Processing Unit	8
4.3	Motor Driver Circuit	8
4.4	Sensor Connections	9
4.5	Circuit Schematics	9
4.6	Camera Interface	9
5	System Architecture Overview	10
5.1	Hardware-Software Interface	10
5.2	Software Layer Architecture	10
5.3	Dual-Core Processing Strategy	10
5.4	Development Environment	11
6	Motor Control Architecture, Sensor Fusion and State Estimation	11
6.1	Motor Control Architecture	11
6.1.1	PWM Configuration	11
6.1.2	Motor Control Primitives	11
6.1.3	Asymmetric PWM Control	11
6.2	Sensor Fusion and State Estimation	12
6.2.1	IR Sensor Array Processing	12
7	Obstacle Avoidance and Navigation Control	12
7.1	Avoidance Sequence	12
7.2	Line Following Algorithm	12
7.2.1	Priority-Based Control Logic	12
7.2.2	Junction Handling	12
7.3	Deviation and Recovery Protocol	13
8	Image Detection and Identification	13
8.1	Image Processing Pipeline	13

8.2	Camera Data Acquisition	13
8.3	Object Detection Method	14
8.4	Object Identification Process	14
8.5	Detection Output	14
9	Web Dashboard	14
9.1	Dashboard Overview	14
9.2	Live Camera Feed and Object Identification	15
9.3	Timer Display and Communication with Rover	15
10	System Integration	15
10.1	Hardware and Software Integration	15
10.2	System Data Flow	16
10.3	Communication Protocols	16
11	Bill of Materials	17
12	Conclusion	17

1 Introduction

Autonomous robotic systems are increasingly being used to perform tasks in structured environments with minimal human intervention. In robotics competitions and real-world applications, such systems must demonstrate reliable navigation, perception, and decision-making capabilities while operating within constrained environments. The development of an autonomous rover that can navigate a predefined path, avoid obstacles, and recognize visual information in real time represents an important challenge in robotics and computer vision.

This project focuses on designing and developing a compact autonomous rover capable of performing navigation and image recognition tasks simultaneously. The rover is required to operate completely autonomously using onboard sensors, processing units, and communication modules. It must also transmit recognition results to an external interface, demonstrating integration between robotics hardware, computer vision algorithms, and wireless communication systems.

1.1 Problem Statement

The objective of this challenge is to design and implement a fully autonomous rover capable of navigating a predefined arena while performing real-time image detection and identification. The rover must follow a symmetric floor-marked path, dynamically detect and avoid obstacles, and identify multiple image cards placed along the path.

During operation, the rover must transmit the detected image information wirelessly to a laptop through a web-based dashboard displaying the live camera feed, identified object name, and a timer. The robot must complete two autonomous runs—from point A to B and then B to A—within a limited time while maintaining reliable navigation, perception, and communication throughout the mission.

1.2 Objectives

The primary objectives of the proposed autonomous rover system are:

- Design and develop a compact rover capable of fully autonomous navigation in a predefined arena.
- Implement reliable path-following with obstacle detection and avoidance.
- Develop an onboard vision system for real-time detection and identification of image cards.
- Enable wireless communication to transmit detection results to a web-based dashboard.
- Ensure stable and repeatable system performance within the given operational constraints.

Operational Constraints

Parameter	Constraint
Maximum robot dimensions	25 × 25 × 25 cm
Maximum allowed weight	3 kg
Run duration	Maximum 5 minutes per run
Allowed controllers	Arduino, ESP32/ESP8266, Raspberry Pi
External computation	Not allowed (processing must be onboard)
Path deviation	Must return to path within 10 seconds (30 seconds total)

Table 1: System Constraints

2 System Overview

2.1 Overall System Description

The proposed system is an autonomous rover designed to navigate a predefined arena while performing real-time image detection and identification. The rover integrates two primary subsystems: a navigation system responsible for path following and a perception system responsible for image recognition.

The navigation subsystem is built around an ESP32 microcontroller which processes data from an array of five infrared (IR) sensors to detect the path markings on the arena floor. The rover follows a three-wheel configuration consisting of two rear drive wheels powered by N20 encoder motors and a front caster wheel that provides stability and smooth movement.

For the perception subsystem, a Raspberry Pi is used along with two Raspberry Pi camera modules mounted on both sides of the rover. These cameras capture images of the cards placed along the arena. The captured frames are processed using a trained YOLOv11n object detection model to identify the objects in real time. The system then transmits the detected results and live camera feed to a web-based dashboard where the identified object name and timer are displayed.

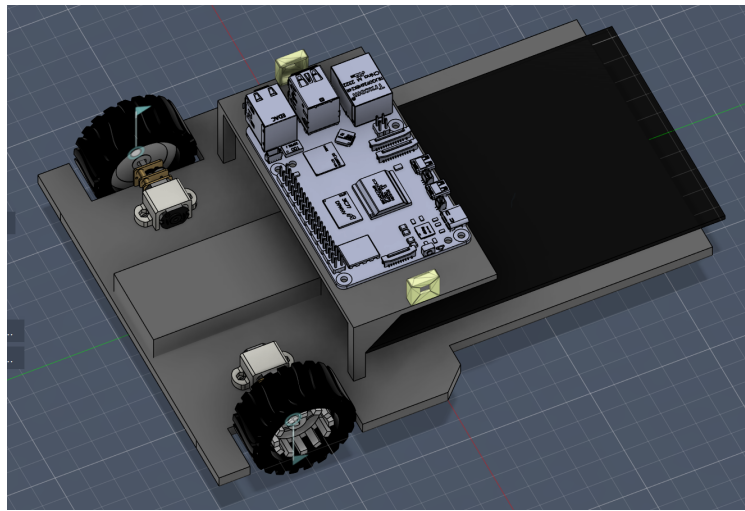


Figure 1: Autonomous Rover Prototype

2.2 Working Principle of the Rover

The rover operates through coordinated interaction between the navigation and perception subsystems. During operation, the IR sensor array continuously detects the floor markings of the arena and provides feedback to the ESP32 controller. Based on the sensor readings, the controller adjusts the speed of the two rear motors to maintain alignment with the path and perform turns at junctions.

At the same time, the Raspberry Pi processes the video streams captured from the two cameras mounted on the sides of the rover. As the rover moves along the path, the cameras capture images of the cards placed around the arena. These images are processed using the YOLOv11n model to detect and classify the objects present on the cards.

Once an object is identified, the detection result is transmitted wirelessly to the dashboard where the live camera feed, detected object name, and timer are displayed.

2.3 System Architecture

The system architecture is divided into two primary modules: the navigation module and the vision processing module.

The navigation module consists of the ESP32 microcontroller, IR sensor array, motor driver, and N20 encoder motors. This module is responsible for path detection, motor control, and maintaining stable movement of the rover along the track.

The vision processing module consists of the Raspberry Pi, dual camera setup, and the trained YOLOv11n detection model. The Raspberry Pi captures images, performs object detection using the trained neural network, and communicates the results to the external dashboard.

Both modules operate simultaneously, allowing the rover to perform navigation and image recognition tasks in parallel.

2.4 Hardware–Software Interaction

The hardware and software components of the rover interact closely to achieve autonomous operation. The ESP32 microcontroller manages the low-level hardware operations such as reading sensor inputs and controlling the motor drivers.

The Raspberry Pi performs high-level processing tasks including image acquisition, object detection, and communication with the external dashboard. The YOLOv11n model processes captured images and identifies the objects in real time.

After detection, the Raspberry Pi transmits the results to the dashboard through Wi-Fi where the live feed, object name, and system timer are displayed. Additionally, detection data can be stored for later analysis and system evaluation.

3 Hardware Design

3.1 Motor and Drive System

The rover uses a **differential drive system** powered by two DC encoder motors. This configuration enables the robot to move forward, reverse, and turn by varying the relative speed of the left and right motors.

Motor control is achieved using the **TB6612FNG motor driver**, which allows efficient bidirectional control and PWM speed modulation.

Motor Driver Connections

ESP32 Pin	Motor Driver Pin	Function
GPIO 26	AIN1	Left motor direction control
GPIO 27	AIN2	Left motor direction control
GPIO 25	PWMA	Left motor speed control
GPIO 14	BIN1	Right motor direction control
GPIO 32	BIN2	Right motor direction control
GPIO 33	PWMB	Right motor speed control
GPIO 12	STBY	Driver standby control

Table 2: Motor driver pin connections

The TB6612FNG driver outputs (A01/A02 and B01/B02) are connected to the left and right motors respectively. The wires of the right motor were physically swapped to correct its rotation direction.

PWM signals generated by the ESP32 regulate motor speed, allowing precise adjustments required for line following and navigation.

3.2 Battery and Power System

The robot operates entirely on **onboard battery power**, as required by the competition rules. The power system provides stable voltage levels to the ESP32 controller, sensors, camera module, and motor driver.

The battery supplies power to the **TB6612FNG motor driver**, which directly drives the DC motors. A regulated supply powers the **ESP32 DevKitC-32E microcontroller** and other electronic components.

Proper power distribution ensures stable sensor operation, reliable motor performance, and minimal electrical noise affecting sensor readings. Voltage regulation and proper grounding were implemented to prevent fluctuations that could affect the control system.

3.3 Sensors and Perception Hardware

The rover uses multiple sensors to perceive its environment and navigate the arena.

IR Line Sensors

Five **infrared reflective sensors** are used for line detection. These sensors detect the contrast between the white arena floor and the black path markings.

The sensor array consists of:

- Far Left IR sensor
- Left IR sensor
- Middle IR sensor
- Right IR sensor
- Far Right IR sensor
- Object IR sensor

Sensor readings follow this logic:

- White surface → 0
- Black line → 1

Sensor Pin Configuration

ESP32 Pin	Sensor
GPIO 22	Far Left IR
GPIO 21	Left IR
GPIO 5	Middle IR
GPIO 23	Right IR
GPIO 4	Far Right IR

Table 3: IR sensor pin configuration

This arrangement enables the robot to detect the line position relative to the center of the rover and perform real-time course correction.

3.4 Camera and Image Processing System

A **Raspberry Pi** is used for image detection and processing. The camera module is connected directly to the Raspberry Pi, which performs onboard image recognition of the cards placed around the arena.

These cards measure approximately **250 mm × 300 mm** and are positioned along the course. As the robot approaches a card, the camera captures an image frame which is processed by the Raspberry Pi to identify the displayed image.

Once the image is detected, the Raspberry Pi sends the identified image label to the monitoring laptop through the wireless network.

Using the Raspberry Pi for vision processing allows computationally intensive image recognition tasks to run independently from the control system.

3.5 Communication Modules

Wireless communication between the robot and the monitoring system is achieved using the onboard networking capabilities of the **Raspberry Pi**.

The Raspberry Pi handles the transmission of vision-related data, including the camera stream and the results of the image recognition process. The processed information is sent to the monitoring laptop over a wireless network.

The Raspberry Pi transmits the following data:

- Live camera feed captured by the onboard camera
- Identified image card name from the vision algorithm

The **ESP32 DevKitC-32E** operates primarily as the real-time control unit and is responsible for sensor processing and motor control. Communication between the ESP32 and Raspberry Pi enables coordination between the navigation system and the vision system.

The transmitted information is displayed on a **web-based dashboard**, allowing real-time monitoring of the robot's progress during the competition.

3.6 Hardware Integration

The rover uses a **distributed processing architecture** consisting of two main controllers:

- **ESP32 DevKitC-32E** – Responsible for real-time control tasks such as:
 - Line following using the IR sensor array
 - Obstacle detection using the ultrasonic sensor
 - Motor control through the TB6612FNG motor driver
- **Raspberry Pi** – Responsible for high-level vision tasks such as:
 - Capturing images using the onboard camera
 - Performing image recognition of arena cards
 - Transmitting detected image names to the monitoring laptop

This separation ensures that real-time navigation tasks run reliably on the ESP32, while computationally intensive image processing is handled by the Raspberry Pi.

4 Circuit Design

4.1 Circuit Architecture

The robot's electronic system is designed around the ESP32 microcontroller, which acts as the central processing unit for sensor processing and motor control. The circuit integrates multiple subsystems including line detection sensors, a motor driver for locomotion, an ultrasonic sensor for obstacle detection, a Raspberry Pi for image processing, and a regulated power supply.

The power is provided through a battery source which is stepped down using a buck converter to supply a stable 5V rail. The ESP32 module operates as the main controller for real-time control tasks, while the Raspberry Pi performs high-level image processing tasks such as object detection.

The architecture consists of the following major blocks:

- ESP32 Microcontroller
- Raspberry Pi (Image Processing Unit)
- Motor Driver Module

- Infrared (IR) Line Sensors
- Camera Module
- Power Regulation (Buck Converter)
- Dual Encoder Motors

The ESP32 reads sensor data, processes the control logic, and generates PWM signals to control the speed and direction of the motors through the motor driver. The Raspberry Pi processes image data from the camera and can communicate with the ESP32 to assist in advanced navigation decisions.

4.2 Microcontroller / Processing Unit

The ESP32-DEVKITC-32E microcontroller is used as the primary processing unit for real-time control operations. It provides multiple GPIO pins, hardware PWM capability, and sufficient processing power for robotic control.

The ESP32 is responsible for:

- Reading line sensor inputs
- identifying object with the help of ir sensor
- Processing navigation algorithms
- Generating PWM signals for motor speed control
- Communicating with the Raspberry Pi for image-based decisions

The following GPIO pins are used for the IR sensors:

Sensor	ESP32 GPIO Pin
Far Left IR Sensor	GPIO 22
Left IR Sensor	GPIO 21
Middle IR Sensor	GPIO 5
Right IR Sensor	GPIO 23
Far Right IR Sensor	GPIO 4
object IR sensor	GPIO 19

The ESP32 generates PWM signals with a frequency of 1000 Hz and an 8-bit resolution to control motor speed.

4.3 Motor Driver Circuit

A dual H-Bridge motor driver is used to control two DC encoder motors. The motor driver receives PWM and direction signals from the ESP32.

The motor driver pins are connected as follows:

- AIN1 : GPIO 26
- AIN2 : GPIO 27
- PWMA : GPIO 25
- BIN1 : GPIO 14
- BIN2 : GPIO 32
- PWMB : GPIO 33
- STBY : GPIO 12

The PWM signals control the motor speed while the direction pins determine the rotation direction. The STBY pin is used to enable or disable the motor driver.

The motor driver controls two motors:

- Left Encoder Motor
- Right Encoder Motor

Each motor is connected to the corresponding output terminals of the driver.

4.4 Sensor Connections

Five infrared line sensors are used for line following. These sensors detect the contrast between the track and the background surface.

The sensors are arranged in a horizontal array at the front of the robot:

- Far Left Sensor (FL)
- Left Sensor (L)
- Middle Sensor (M)
- Right Sensor (R)
- Far Right Sensor (FR)

These sensors provide digital signals to the ESP32 indicating whether the sensor is detecting the line or the background.

4.5 Circuit Schematics

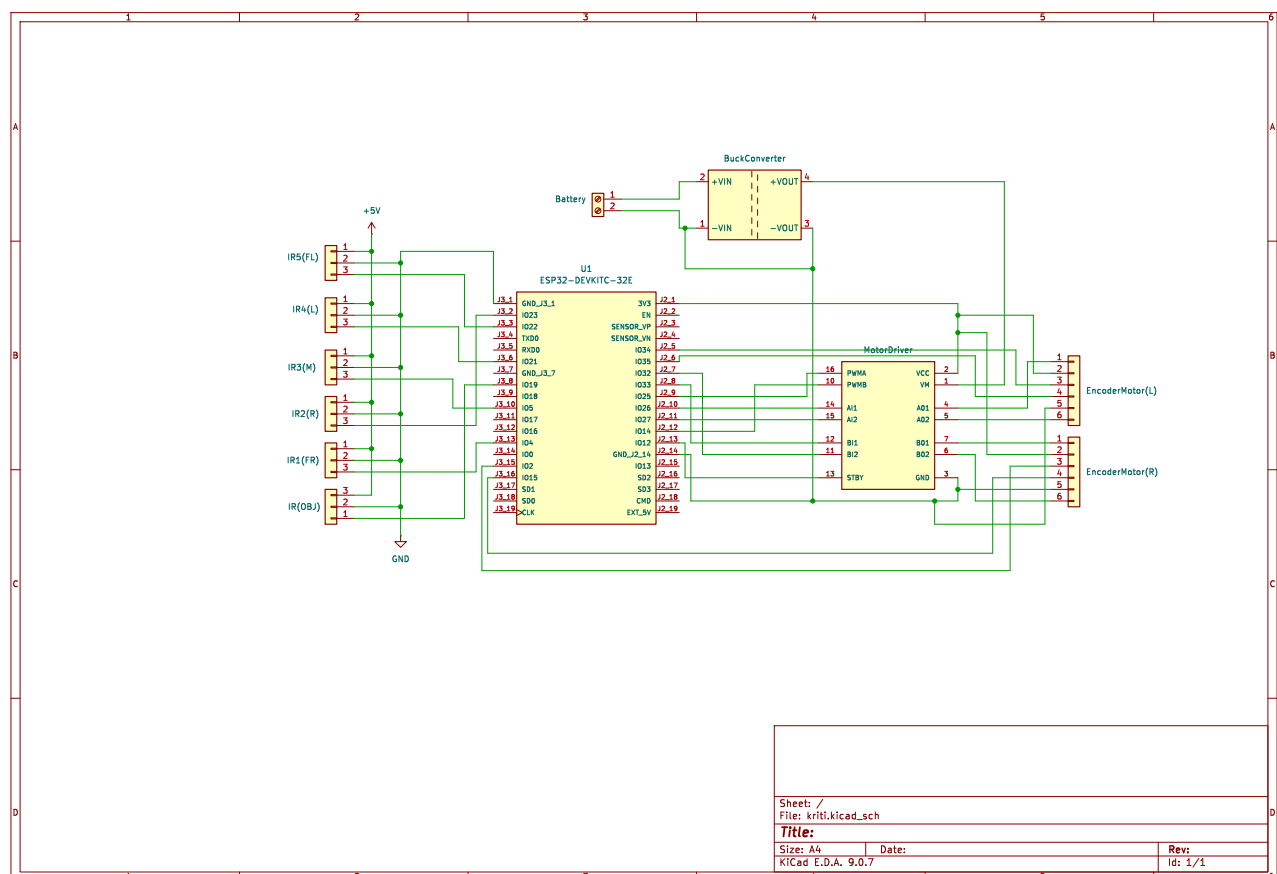


Figure 2: Electric circuit diagram

4.6 Camera Interface

A camera module is connected to the Raspberry Pi for image acquisition and processing. The Raspberry Pi acts as a dedicated image processing unit capable of running computer vision algorithms for object detection

and environmental analysis.

The camera captures images of the robot’s surroundings and sends them to the Raspberry Pi, where image processing techniques such as edge detection, object recognition, or color tracking can be implemented using libraries like OpenCV.

The Raspberry Pi processes the visual data and communicates relevant navigation decisions to the ESP32 using serial communication. The ESP32 then adjusts motor control accordingly.

This hybrid architecture allows computationally intensive tasks such as image processing to be handled by the Raspberry Pi, while the ESP32 focuses on real-time control of sensors and actuators.

5 System Architecture Overview

The autonomous rover operates on a distributed computing architecture comprising two primary processing units: the **ESP32 DevKitC-32E** for real-time navigation and obstacle avoidance, and the **Raspberry Pi** for image processing and WiFi communication. This dual-processor design ensures deterministic response times for critical control tasks while leveraging the computational power of the Raspberry Pi for computer vision operations.

5.1 Hardware-Software Interface

The software architecture is built upon the following hardware foundation:

Table 4: Hardware-Software Mapping

Hardware Component	Software Module	Functionality
ESP32 DevKitC-32E	Navigation Control System	Real-time line following, obstacle avoidance
Raspberry Pi 4	Image Detection Pipeline	Object recognition, WiFi communication
TB6612FNG	Motor Control Driver	PWM-based differential drive
5× IR Sensors	Line Detection Algorithm	Path tracking and junction detection
HC-SR04	Obstacle Detection Task	Distance measurement (Core 0)
Encoder Motors	Velocity Control Loop	Closed-loop speed regulation
Camera Module	OpenCV/Image Processing	A4 card identification

5.2 Software Layer Architecture

5.3 Dual-Core Processing Strategy

The ESP32 utilizes its dual-core architecture through FreeRTOS task pinning:

- **Core 0 (Protocol CPU):** Dedicated to ultrasonic sensor polling via the dedicated obstacle detection task. This ensures consistent 33Hz distance measurement updates without interference from control algorithms.
- **Core 1 (Application CPU):** Executes the main control loop, handling IR sensor reading, motor control, and state machine transitions for line following and obstacle avoidance.

Inter-core communication is implemented through a mutex-protected shared variable ensuring thread-safe access to obstacle distance data.

5.4 Development Environment

Table 5: Development Toolchain

Component	Specification
IDE	Arduino IDE 2.x / PlatformIO
Framework	Arduino-ESP32 Core 3.0+
RTOS	FreeRTOS (built-in)
Raspberry Pi OS	Raspberry Pi OS Lite (64-bit)
Vision Library	OpenCV 4.x
Web Framework	Flask (Python)
Communication Protocol	HTTP/WebSocket over WiFi

6 Motor Control Architecture, Sensor Fusion and State Estimation

6.1 Motor Control Architecture

The rover employs a differential drive configuration controlled by the TB6612FNG motor driver. The control system implements a dual-loop architecture with open-loop speed control for basic navigation and closed-loop encoder feedback for precise maneuvering.

6.1.1 PWM Configuration

The motor control system utilizes the LEDC peripheral for precise PWM generation with the following specifications:

Table 6: PWM Configuration Parameters

Parameter	Value	Purpose
Frequency	1 kHz	Optimal torque response
Resolution	8-bit (0–255)	Granular speed control
Base Speed	250/255	Standard cruising velocity
Turn Speed	150/255	Pivot rotation speed
Correction Speed	170/255	Gentle curve correction

6.1.2 Motor Control Primitives

The system implements four fundamental motor control functions:

- 1. **Forward Motion:** Both motors rotate clockwise at equal speed for straight-line movement.
- 2. **Turn Left:** Left motor reverses while right motor moves forward, creating pivot rotation.
- 3. **Turn Right:** Right motor reverses while left motor moves forward, creating pivot rotation.
- 4. **Stop:** Both motors brake to halt immediately.

6.1.3 Asymmetric PWM Control

For gentle curve following without sharp turns, the system employs asymmetric PWM duty cycles:

- **Slight Left:** Left motor at 25% speed, right motor at 100% speed
- **Slight Right:** Left motor at 100% speed, right motor at 25% speed

6.2 Sensor Fusion and State Estimation

6.2.1 IR Sensor Array Processing

The five-sensor array provides discrete line position detection. Sensor states are processed using a weighted voting algorithm with the following priority encoding:

Table 7: IR Sensor State Encoding and Actions

FL	L	M	R	FR	Action	Priority
0	X	X	X	1	Turn Left (Junction)	High
1	X	X	X	0	Turn Right (Junction)	High
X	0	1	0	X	Slight Left	Normal
X	1	0	0	X	Forward	Normal
X	0	0	1	X	Slight Right	Normal
0	0	0	0	0	Search Pattern	Recovery

7 Obstacle Avoidance and Navigation Control

The avoidance maneuver is a pre-programmed sequence based on empirical timing calibrated for the 25cm×25cm×25cm robot dimensions and 250mm obstacle size specified in the problem statement.

7.1 Avoidance Sequence

The maneuver consists of seven sequential phases:

1. **Initial Left Turn (320ms):** Rotate 45° left to angle away from obstacle
2. **Forward Displacement 1 (300ms):** Move forward to clear obstacle width
3. **Right Turn 1 (350ms):** Rotate 90° right to parallel obstacle
4. **Forward Passage (400ms):** Move past obstacle length
5. **Right Turn 2 (350ms):** Rotate 90° right to face original path
6. **Line Search:** Move forward until center IR sensor detects line
7. **Realignment (100ms):** Brief left turn to straighten orientation

Total maneuver time: approximately 1.82 seconds plus line reacquisition time.

7.2 Line Following Algorithm

The navigation system implements a **Bang-Bang control** strategy with weighted sensor response. The algorithm prioritizes junction detection over fine line tracking to ensure correct path decisions at intersections.

7.2.1 Priority-Based Control Logic

The main control loop operates with three priority levels:

Highest Priority: Obstacle Avoidance Immediate execution when ultrasonic sensor detects obstacle within 5cm threshold. Triggers complete avoidance sequence.

High Priority: Junction Detection Activated when far-left or far-right IR sensors detect line, indicating intersection approach. Executes 90° turn in appropriate direction with center sensor alignment verification.

Normal Priority: Line Following Standard operation using three-center sensors for path tracking with slight left/right corrections.

7.2.2 Junction Handling

When a junction is detected (FL=0 or FR=0):

1. Execute pivot turn in detected direction

2. Continue turning until center IR sensor detects line (alignment verification)
3. Resume normal line following

This ensures the robot properly aligns with the new path segment before continuing.

7.3 Deviation and Recovery Protocol

The algorithm handles temporary path deviation as per competition rules (10 seconds per-instance, 30 seconds total off-path limits):

Table 8: Deviation Monitoring Parameters

Parameter	Specification
Per-Instance Limit	10 seconds maximum
Total Deviation Limit	30 seconds cumulative
Detection Method	Center + adjacent IR sensors all HIGH (white surface)
Recovery Action	Spiral search pattern with increasing radius
Violation Action	Emergency stop, mission failure logged

8 Image Detection and Identification

8.1 Image Processing Pipeline

The image detection system is implemented on a Raspberry Pi using a trained YOLOv11n object detection model. The system processes video streams from two cameras mounted on the rover. Each frame captured from the cameras is passed through a preprocessing and inference pipeline where objects are detected and classified in real time.

The captured frames are resized to a resolution of 320×240 pixels to maintain a balance between computational efficiency and detection accuracy. Frames are processed at regular intervals to reduce computational load while ensuring continuous monitoring of the environment.

The YOLO model predicts bounding boxes, class IDs, and confidence scores for objects detected within the frame. These detections are then annotated directly on the video stream and transmitted to the dashboard for visualization.

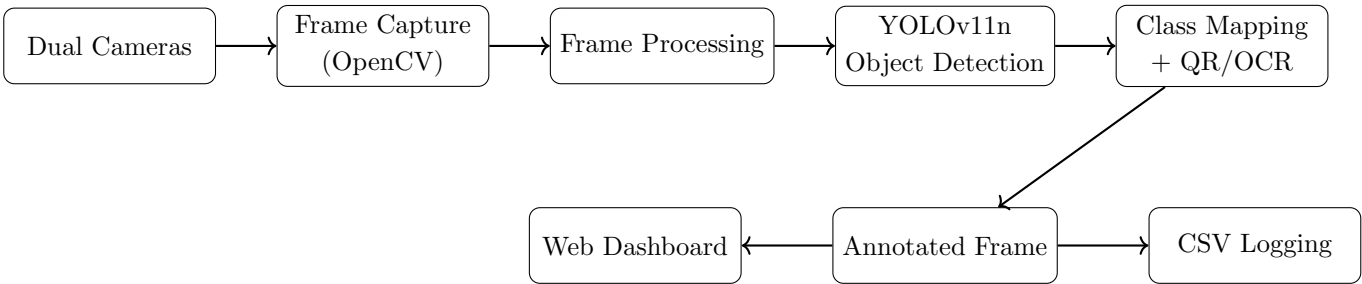


Figure 3: Image Detection Pipeline of the Autonomous Rover

8.2 Camera Data Acquisition

The rover uses two Raspberry Pi camera modules placed on opposite sides of the robot to increase the field of view. The cameras capture continuous frames which are processed using OpenCV before being fed into the object detection model.

The camera frames are converted into a suitable format and stored in a queue for processing. This design allows simultaneous frame capture and inference without blocking the system pipeline. The processed frames are then streamed through a Flask-based web server as an MJPEG video feed, enabling real-time monitoring through a web dashboard.

8.3 Object Detection Method

Object detection is performed using the YOLO (You Only Look Once) deep learning model trained specifically for the task environment. The trained model identifies multiple object classes including logos, objects, animals, vehicles, QR codes, and number plates.

The detection algorithm predicts bounding boxes around objects and assigns a class label along with a confidence score. A confidence threshold is applied to filter out low-confidence predictions and ensure reliable detections.

Once the objects are detected, bounding boxes and labels are drawn on the video frames using OpenCV. Each detection includes the class name and confidence score which are displayed on the dashboard.

8.4 Object Identification Process

After detection, the identified objects are mapped to predefined class names. The system maintains a dictionary that associates class IDs with object names. This mapping allows the detected class index from the model to be converted into a human-readable label.

For specific object classes such as QR codes and number plates, additional processing steps are performed. QR codes are decoded using OpenCV's QR detection module, while number plates are processed using an OCR-based approach to extract alphanumeric characters.

The final detection result consists of the object class name, confidence score, and any decoded information when applicable.

8.5 Detection Output

The detection results are displayed on the web dashboard in real time. Each detected object is logged along with the following information:

- Timestamp of detection
- Camera source
- Object class ID
- Object class name
- Detection confidence

All detection results are stored in a CSV file for later analysis. This logging mechanism ensures that the system maintains a record of all recognized objects during operation. The detection output is simultaneously visualized on the dashboard and recorded in the dataset for evaluation and debugging purposes.

9 Web Dashboard

9.1 Dashboard Overview

A web-based dashboard was developed to monitor the rover's perception system in real time. The dashboard receives data from the Raspberry Pi and displays the live camera feed along with object detection results. It also maintains a record of detected objects and system statistics during operation. The interface provides a simple way to visualize the rover's environment and verify detection performance during the run.

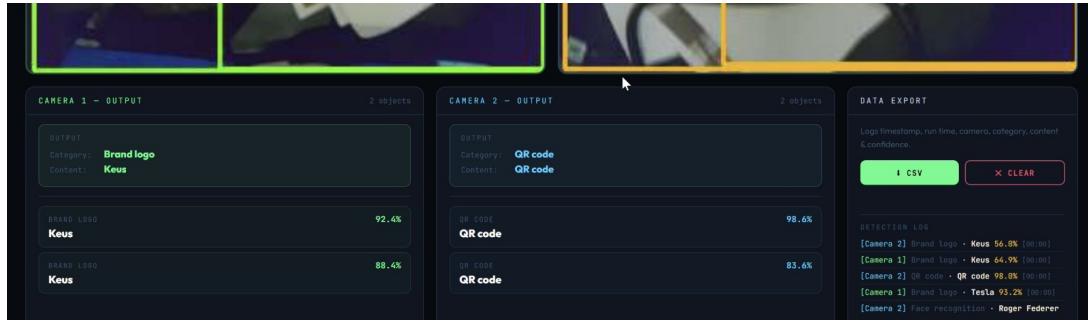


Figure 4: Web Dashboard Interface

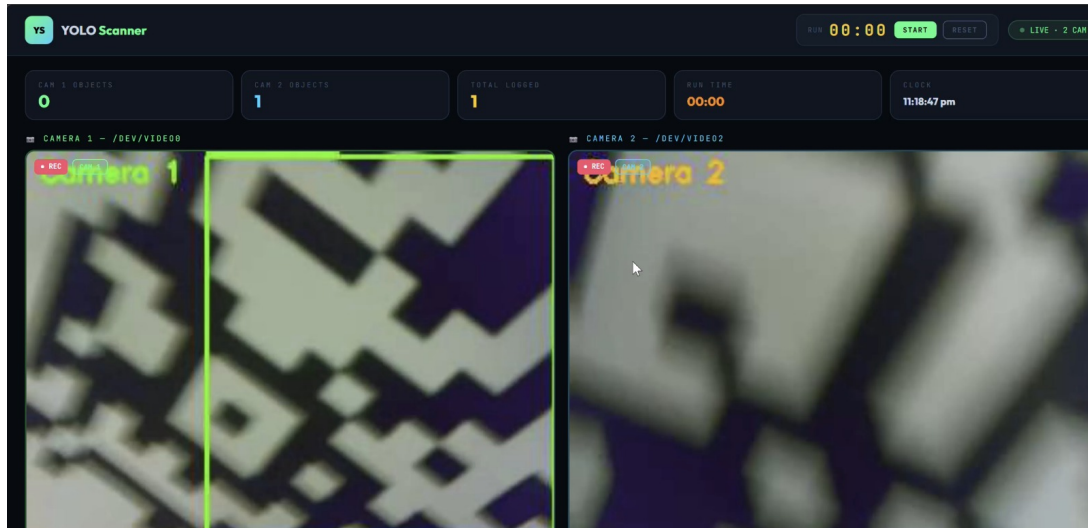


Figure 5: Web Dashboard Interface

9.2 Live Camera Feed and Object Identification

The dashboard displays the live video streams from two Raspberry Pi cameras mounted on both sides of the rover. These streams are processed using the YOLOv11n model, which detects and classifies objects in real time. Detected objects are displayed in a list along with their class name and confidence score. The system updates the detection panel continuously as new objects are identified.

9.3 Timer Display and Communication with Rover

The dashboard includes a timer that tracks the system runtime during operation. The rover communicates with the dashboard through a Wi-Fi connection established by the Raspberry Pi. Detection data, including timestamp, camera source, object class, and confidence score, is logged and can be exported as a CSV file for later analysis.

10 System Integration

10.1 Hardware and Software Integration

The autonomous rover system consists of two main subsystems: the navigation subsystem and the vision subsystem. These two systems operate independently while sharing the same power source.

The navigation subsystem is built around an ESP32 microcontroller connected to five infrared (IR) sensors used for line detection. The ESP32 processes the sensor inputs and generates control signals for the motor driver to control the two N20 encoder motors. This subsystem is responsible for the real-time movement of the robot and ensures that the rover follows the predefined path in the arena.

The vision subsystem is implemented using a Raspberry Pi placed on the robot. The Raspberry Pi is connected to two camera modules which capture images of the environment. The images are processed

using the YOLOv11n object detection model to identify the objects placed in the arena. The Raspberry Pi operates independently from the ESP32 and performs all image processing tasks locally.

Both subsystems are powered by the same battery pack but do not communicate directly with each other. This separation ensures that the real-time navigation control performed by the ESP32 is not affected by the computational load of the image processing tasks on the Raspberry Pi.

10.2 System Data Flow

The system data flow is divided into two independent pipelines corresponding to the navigation and perception subsystems.

In the navigation pipeline, the IR sensors continuously detect the line markings on the arena surface and send digital signals to the ESP32 microcontroller. Based on these signals, the ESP32 determines the position of the line relative to the robot and adjusts the motor speeds accordingly using PWM signals. This process allows the robot to follow the path and handle turns or junctions.

In the perception pipeline, the cameras connected to the Raspberry Pi continuously capture frames from the surroundings. These frames are processed using the YOLOv11n model which detects and classifies objects present in the scene. The detected objects are displayed on the web dashboard along with the live video feed. Detection results are also logged into a CSV file for later analysis and evaluation.

10.3 Communication Protocols

The ESP32 and Raspberry Pi do not communicate directly in the current system implementation. The ESP32 handles navigation tasks independently, while the Raspberry Pi performs image detection and data visualization.

The Raspberry Pi communicates with the web dashboard using a Wi-Fi connection through a Flask-based web server. The dashboard streams the live camera feed using an MJPEG video stream and periodically retrieves detection data using HTTP requests.

This architecture simplifies system design by separating navigation control from image processing, allowing each subsystem to operate efficiently without interfering with the other.

11 Bill of Materials

Table 9: Bill of Materials

S.No	Component / Device	Quantity
1	Infrared Obstacle Avoidance Sensor Module (Active Low)	6
2	TB6612FNG Dual Motor Driver Module	1
3	12V Encoder DC Gear Motor	2
4	Robot Wheels	2
5	ESP32 Development Board (ESP-WROOM-32)	1
6	Raspberry Pi 5 Single Board Computer (8GB RAM)	1
7	Raspberry Pi Camera Module V1.3 (5MP)	2
8	18650 Li-ion Battery Pack	1
9	Jumper Wires / Connectors	1 Set
10	Robot Chassis / Frame	1
11	perfboard	1 Set

12 Conclusion

This project presented the design and development of an autonomous rover capable of performing navigation and real-time image detection tasks within a constrained arena environment. The system successfully integrates embedded control, computer vision, and web-based monitoring to achieve reliable autonomous operation.

The navigation subsystem is implemented using an ESP32 microcontroller with a five-sensor infrared array for accurate line following and path tracking. The rover uses a differential drive configuration powered by N20 encoder motors, enabling stable movement and precise turning during navigation.

For the perception subsystem, a Raspberry Pi equipped with dual camera modules performs real-time object detection using a trained YOLOv11n model. The detected objects are displayed through a web-based dashboard that streams live camera feeds and provides identification results along with system timing information.

The system architecture separates navigation and vision processing tasks, allowing the ESP32 to handle real-time control while the Raspberry Pi manages computationally intensive image detection operations. Both subsystems operate using a shared power supply while functioning independently to ensure reliable performance.

Overall, the developed system demonstrates the successful integration of robotics hardware, computer vision algorithms, and network-based monitoring to create a functional autonomous rover capable of completing the tasks specified in the challenge.